GRUPO CORDILLERA

Plataforma de Monitoreo Inteligente

para el Desempeño Organizacional

MICROSERVICIOS

Arquitectura con Java / Spring Boot
Docker y Docker Compose

Integrantes:	Tomas Gonzalez Borje
	Luis Alejandro Rojas Gil

Curso:	Arquitectura de Software
Fecha:	30 de marzo de 2026
Version:	1.0

0 Índice

1. Introduccion **3**

2. Requerimientos Funcionales y No Funcionales **4**
2.1. Requerimientos Funcionales (RF) 4
2.2. Requerimientos No Funcionales (RNF) 5
2.3. Justificacion de los Requerimientos 6

3. Microservicios **7**

4. Diagrama de Arquitectura General **7**

5. Patrones de Diseño Aplicados **8**
5.1. Repository Pattern 8
5.2. Factory Method 9
5.3. Circuit Breaker 9

6. Especificación de Cada Servicio **10**
6.1. Auth Service 10
6.1.1. Responsabilidad 10
6.1.2. Tecnologias y Justificacion 10
6.1.3. Diagrama de Flujo: Autenticacion y Emision de Token 11
6.1.4. Dockerfile 12
6.2. Data Ingestion Service 12
6.2.1. Responsabilidad 12
6.2.2. Tecnologias y Justificacion 12
6.2.3. Diagrama de Flujo: Ciclo de Ingesta con Factory Method y Circuit Breaker 13
6.2.4. Dockerfile 14
6.3. KPI Engine Service 14
6.3.1. Responsabilidad 14
6.3.2. Tecnologias y Justificacion 14
6.3.3. Diagrama de Flujo: Calculo de KPI y Deteccion de Alertas . . . 15
6.3.4. Dockerfile 16
6.4. Report Service 16
6.4.1. Responsabilidad 16
6.4.2. Tecnologias y Justificacion 16
6.4.3. Diagrama de Flujo: Generacion de Reporte 17
6.4.4. Dockerfile 18

7. Orquestacion con Docker Compose **19**

8. Diagrama BPMN: Flujo Completo entre los Cuatro Servicios **22**

9. Modelo de Datos por Servicio **22**

10. Observabilidad y Monitoreo **23**
10.1. Logs Estructurados 23
10.2. Correlation ID 24
10.3. Metricas con Spring Boot Actuator 24
10.4. Justificacion Tecnica Adicional 25

11. Justificacion de Decisiones Tecnicas **25**
11.1. Por que Java 21 con Spring Boot 3 para todos los servicios 25
11.2. Por que PostgreSQL para todos los servicios 26
11.3. Por que comunicacion sincrona HTTP y no Kafka en esta etapa 26
11.4. Por que JasperReports y no Puppeteer 26

12. Variables de Entorno y Configuracion **27**

13. Conclusion **28**

1 Introduccion

Este documento define la arquitectura de los cuatro microservicios de la plataforma de monitoreo de Grupo Cordillera. Todos los servicios se desarrollan en Java con Spring Boot, se utiliza Docker y se orquestan localmente con Docker Compose.

Alcance de este Informe: Definicion de servicios, tecnologias, patrones de diseño (Repository Pattern, Factory Method, Circuit Breaker), diagramas y configuracion Docker. Las integraciones con bus de eventos (Apache Kafka) y cache distribuida (Redis) se implementaran en la segunda iteracion, cuando se avance a la capa de comunicacion asincrona.

La decision de usar un unico lenguaje y framework (Java + Spring Boot) en todos los servicios no es arbitraria: el equipo conoce el stack, Spring Boot tiene soporte nativo para todos los patrones requeridos, y mantener un solo runtime simplifica la configuracion de Docker y los pipelines de construccion.

2 Requerimientos Funcionales y No Funcionales

Esta sección define los requerimientos funcionales (RF) y no funcionales (RNF) de la plataforma de monitoreo inteligente de Grupo Cordillera. Estos requisitos se derivan directamente del problema identificado: la dispersión de datos, la dependencia de procesos manuales y la falta de visibilidad en tiempo real.

2.1 Requerimientos Funcionales (RF)

ID	Descripción
RF-01	El sistema debe permitir la autenticación de usuarios mediante email y contraseña.
RF-02	El sistema debe generar y validar tokens JWT para el acceso seguro a los servicios.
RF-03	El sistema debe gestionar roles y permisos (RBAC) para controlar el acceso a funcionalidades.
RF-04	El sistema debe integrarse con sistemas externos (POS, ERP, e-commerce, CRM).
RF-05	El sistema debe extraer datos automáticamente desde los sistemas fuente.
RF-06	El sistema debe transformar los datos a un esquema unificado.
RF-07	El sistema debe almacenar los datos consolidados en una base de datos centralizada por servicio.
RF-08	El sistema debe permitir definir indicadores clave de desempeño (KPI).
RF-09	El sistema debe calcular automáticamente los KPI a partir de los datos ingresados.
RF-10	El sistema debe detectar desviaciones en los KPI según umbrales definidos.
RF-11	El sistema debe generar alertas cuando un KPI supere su umbral.
RF-12	El sistema debe exponer una API REST para consultar KPI filtrados por periodo, sucursal y área.
RF-13	El sistema debe permitir la generación de reportes en formato PDF y Excel.
RF-14	El sistema debe permitir la descarga de reportes mediante URL temporal.
RF-15	El sistema debe registrar el historial de reportes generados.
RF-16	El sistema debe permitir la consulta de datos en tiempo cercano a real (near real-time).
RF-17	El sistema debe notificar al motor de KPI cuando se ingresen nuevos datos.
RF-18	El sistema debe validar la integridad de los datos antes de persistirlos.

2.2 Requerimientos No Funcionales (RNF)

ID	Descripción
RNF-01	El sistema debe ser escalable horizontalmente mediante contenedores Docker.
RNF-02	El sistema debe garantizar alta disponibilidad de los servicios críticos.
RNF-03	El sistema debe responder a solicitudes en menos de 2 segundos en condiciones normales.
RNF-04	El sistema debe soportar múltiples usuarios concurrentes sin degradación significativa.
RNF-05	El sistema debe implementar autenticación y autorización segura mediante JWT.
RNF-06	Las contraseñas deben almacenarse utilizando algoritmos de hash seguros (BCrypt).
RNF-07	El sistema debe aislar las bases de datos por microservicio.
RNF-08	El sistema debe ser tolerante a fallos mediante el uso de Circuit Breaker.
RNF-09	El sistema debe registrar logs estructurados para facilitar el monitoreo.
RNF-10	El sistema debe permitir la trazabilidad de solicitudes mediante Correlation ID.
RNF-11	El sistema debe exponer métricas de salud y rendimiento mediante Spring Boot Actuator.
RNF-12	El sistema debe ser desplegable mediante Docker Compose en entorno local.
RNF-13	El sistema debe ser mantenible, siguiendo principios de separación de responsabilidades.
RNF-14	El sistema debe permitir la extensión de nuevos conectores sin modificar código existente (Factory Method).
RNF-15	El sistema debe asegurar consistencia eventual en el procesamiento de datos.
RNF-16	El sistema debe manejar errores de forma controlada sin afectar la operación global.
RNF-17	El sistema debe proteger la comunicación entre servicios mediante HTTPS (en producción).
RNF-18	El sistema debe permitir futura integración con herramientas de monitoreo (Prometheus, Grafana).

2.3 Justificacion de los Requerimientos

Los requerimientos definidos responden directamente a las problemáticas identificadas en Grupo Cordillera.

Los requerimientos funcionales abordan la necesidad de automatizar la recolección, procesamiento y análisis de datos, eliminando la dependencia de procesos manuales. En particular, la definición de módulos separados para ingesta, cálculo de KPI y generación de reportes refleja la arquitectura de microservicios adoptada.

Por otro lado, los requerimientos no funcionales aseguran que la solución sea viable en un entorno real. Características como escalabilidad, tolerancia a fallos y seguridad son fundamentales en sistemas distribuidos donde múltiples servicios interactúan entre sí.

La inclusión de aspectos como observabilidad, trazabilidad y monitoreo desde etapas tempranas permite que la plataforma no solo funcione correctamente, sino que también pueda ser mantenida y evolucionada en el tiempo sin incrementar la complejidad operativa.

Alineacion con la Arquitectura

Cada grupo de requerimientos está directamente asociado a un microservicio:

- Auth Service: RF-01 a RF-03, RNF-05, RNF-06
- Data Ingestion: RF-04 a RF-07, RF-17, RF-18
- KPI Engine: RF-08 a RF-12, RF-10, RF-11
- Report Service: RF-13 a RF-15

Esto asegura coherencia entre los requerimientos y la implementación propuesta.

3 Microservicios

La plataforma se divide en cuatro servicios. Cada uno tiene una sola razon para cambiar, su propia base de datos y un contrato de API documentado.

Criterio de inclusion: Un servicio es si su caida hace que la plataforma no cumpla ningun objetivo del negocio. Sin **Auth**, nadie entra. Sin **Data Ingestion**, no hay datos. Sin **KPI Engine**, no hay indicadores. Sin **Report Service**, la alta direccion no tiene salida de informacion.

Servicio	Stack	Responsabilidad
Auth Service	Java 21 + Spring Boot 3 + Spring Security	Autenticacion de usuarios, emision y validacion de tokens JWT, control de acceso por rol (RBAC).
Data Ingestion Service	Java 21 + Spring Boot 3 + Spring Data JPA	Conexion con sistemas fuente (POS, ERP, e-commerce, CRM, inventario) via conectores configurables, transformacion al esquema unificado y persistencia.
KPI Engine Service	Java 21 + Spring Boot 3 + Spring Data JPA	Calculo de indicadores a partir de los datos consolidados, deteccion de desviaciones y exposicion de resultados via API REST.
Report Service	Java 21 + Spring Boot 3 + JasperReports	Generacion de reportes en PDF y Excel bajo demanda, consultando datos del KPI Engine y del repositorio de datos.

Cuadro 3: Definición de microservicios y sus stacks de desarrollo

4 Diagrama de Arquitectura General

El siguiente diagrama muestra los cuatro servicios, la API Gateway como punto de entrada unico, y las bases de datos individuales de cada servicio. La comunicacion entre servicios en esta etapa es exclusivamente sincrona via HTTP REST.

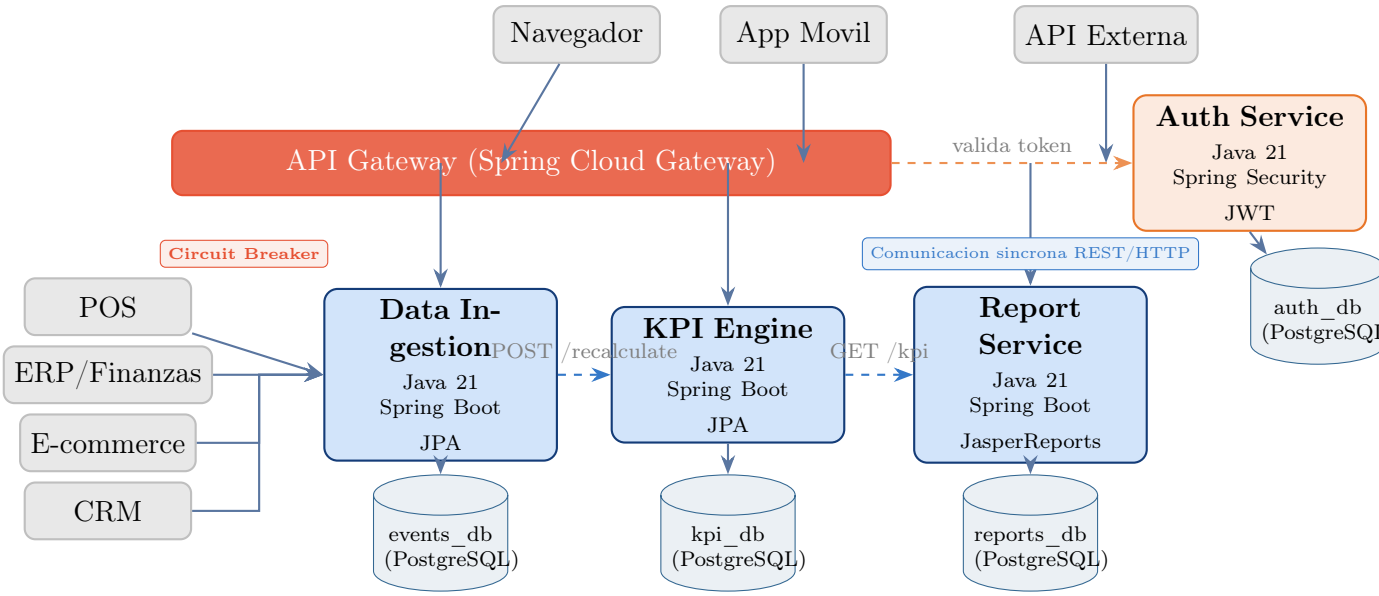

Figura 1: Arquitectura general – cuatro microservicios core con Docker

5 Patrones de Diseño Aplicados

Los tres patrones requeridos se implementan directamente con las herramientas que Spring Boot ya provee, sin librerias externas adicionales.

5.1 Repository Pattern

Repository Pattern – Spring Data JPA

Cada microservicio accede a su base de datos exclusivamente a traves de una interfaz de repositorio. El codigo de negocio invoca metodos como `findById()` o `save()` sin saber si los datos estan en PostgreSQL o en cualquier otro motor. Spring Data JPA genera la implementacion concreta en tiempo de compilacion.

Ejemplo concreto:

```
// KPI Engine Service
public interface KpiDefinitionRepository
        extends JpaRepository<KpiDefinition, UUID> {
    List<KpiDefinition> findByIsActiveTrue();
    Optional<KpiDefinition> findByKpiName(String name);
}
```

Beneficio directo: Cambiar de PostgreSQL a otro motor solo requiere actualizar la dependencia en `pom.xml` y las variables de entorno del contenedor. El codigo de negocio no cambia.

5.2 Factory Method

Factory Method – Conectores de Sistemas Fuente

El Data Ingestion Service necesita conectarse con cinco sistemas distintos (POS, ERP, e-commerce, CRM, inventario), cada uno con su propio protocolo y formato. El Factory Method permite instanciar el conector correcto segun el tipo de sistema, sin que el servicio conozca los detalles de cada integracion.

Ejemplo concreto:

```
// Interfaz comun para todos los conectores
public interface DataConnector {
    List<RawEvent> extractData(SourceSystem system);
    boolean isHealthy();
}

// Factory que decide que implementacion crear
@Component
public class ConnectorFactory {
    public DataConnector create(String connectorType) {
        return switch (connectorType) {
            case "POS"      -> new PosConnector();
            case "ERP"      -> new ErpConnector();
            case "ECOMMERCE" -> new EcommerceConnector();
            case "CRM"      -> new CrmConnector();
            default -> throw new IllegalArgumentException(
                "Tipo de conector desconocido: " + connectorType);
        };
    }
}
```

Beneficio directo: Agregar soporte para un nuevo sistema fuente solo requiere implementar `DataConnector` y registrar el nuevo tipo en el `switch`. No se modifica codigo existente.

5.3 Circuit Breaker

Circuit Breaker – Resilience4j

Cuando un sistema fuente no responde, sin Circuit Breaker el Data Ingestion Service agotaria sus hilos esperando timeouts, bloqueando nuevas solicitudes. Resilience4j implementa el patron directamente sobre Spring Boot con una sola anotacion.

Ejemplo concreto:

```
@Service
public class IngestionService {

    @CircuitBreaker(name = "sourceSystem",
                    fallbackMethod = "useLastSnapshot")
    public List<RawEvent> fetchFromSource(SourceSystem system) {
        DataConnector connector = factory.create(system.getType());
        return connector.extractData(system);
```

```java
    }

    // Se invoca automaticamente cuando el CB esta abierto
    public List<RawEvent> useLastSnapshot(
            SourceSystem system, Exception ex) {
        log.warn("CB abierto para {}. Usando snapshot.", system.getName());
        return snapshotRepository.findLastBy(system.getId());
    }
}
```

Beneficio directo: Un fallo en el POS no bloquea la ingesta de datos del ERP ni impide que los ejecutivos consulten los KPI ya calculados. El dashboard sigue funcionando con los ultimos datos disponibles.

6 Especificación de Cada Servicio

6.1 Auth Service

6.1.1 Responsabilidad

Gestiona identidad y acceso. Autentica usuarios con email y contrasena, firma tokens JWT con clave privada RSA y los valida cuando la API Gateway los presenta. Implementa RBAC para que cada rol solo vea lo que le corresponde: los ejecutivos ven el dashboard completo, los analistas ven datos de sus areas, los administradores gestionan usuarios y conectores.

6.1.2 Tecnologias y Justificacion

Stack: Java 21 + Spring Boot 3 + Spring Security + JJWT

- **Spring Security:** La libreria estandar de seguridad en el ecosistema Spring. Gestiona el ciclo completo de autenticacion y autorizacion con configuracion declarativa, sin necesidad de implementar los filtros HTTP manualmente.
- **JJWT (Java JWT):** Biblioteca ligera para crear y verificar tokens JWT firmados con RS256 (clave asimetrica). Permite que otros servicios validen tokens localmente sin contactar al Auth Service en cada request.
- **PostgreSQL (auth_db):** Almacena usuarios, roles y el log de sesiones activas. La persistencia relacional garantiza integridad referencial entre usuarios y sus asignaciones de rol.
- **BCrypt:** Para el hash de contrasenas. Spring Security lo incluye nativamente y su costo computacional configurable resiste ataques de fuerza bruta por diseno.

6.1.3 Diagrama de Flujo: Autenticacion y Emision de Token

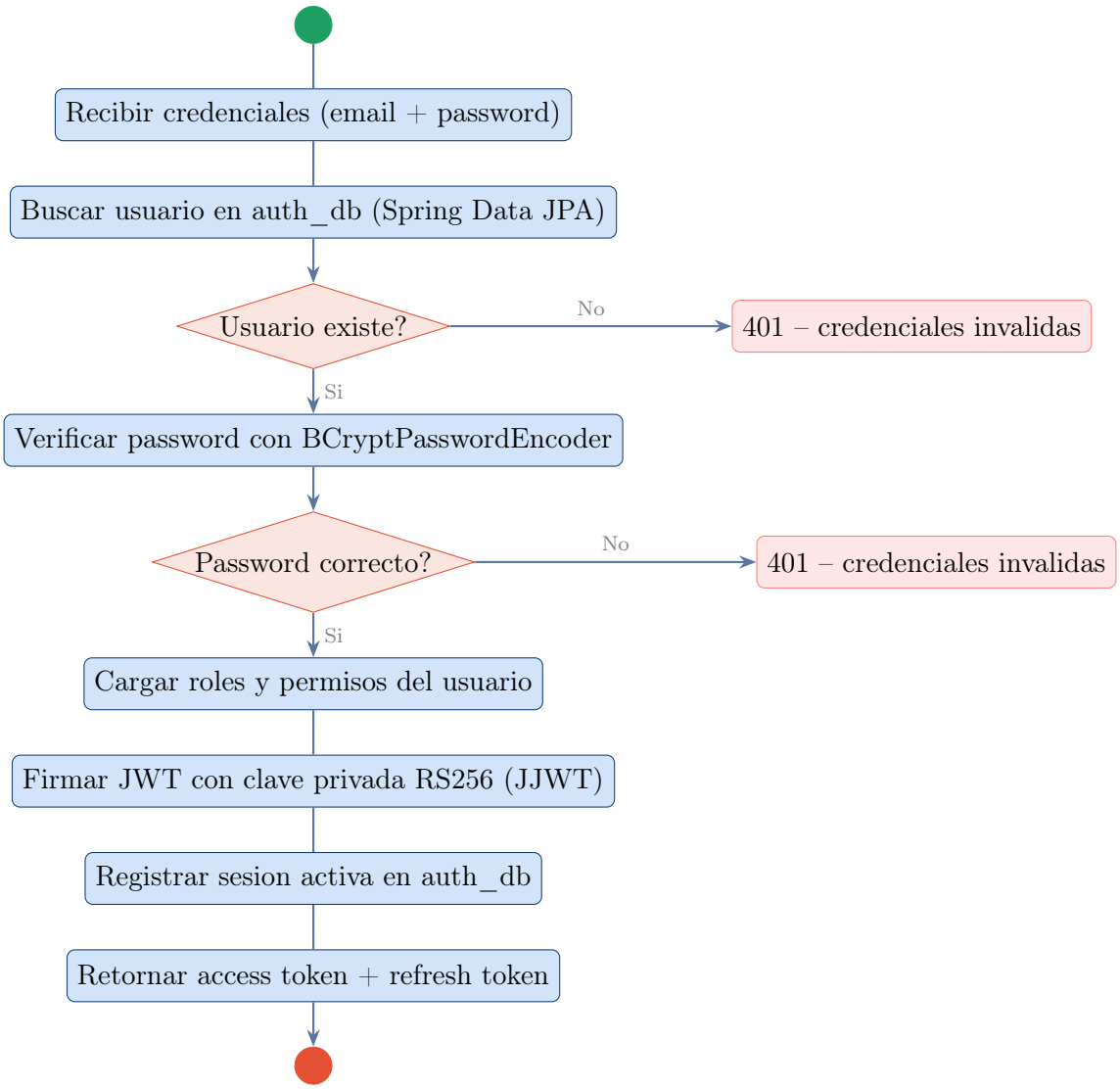

Figura 2: Flujo de autenticacion – Auth Service

6.1.4 Dockerfile

Listing 1: Dockerfile - Auth Service (Java 21 + Spring Boot)

```
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN ./mvnw package -DskipTests --no-transfer-progress

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/auth-service-*.jar app.jar
ENV JAVA_OPTS="-Xms128m -Xmx256m"
EXPOSE 8081
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s \
  CMD wget -qO- http://localhost:8081/actuator/health || exit 1
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

6.2 Data Ingestion Service

6.2.1 Responsabilidad

Se conecta con los cinco sistemas fuente de Grupo Cordillera mediante conectores configurables (Factory Method). Extrae los datos, los transforma al esquema unificado de la plataforma y los persiste en su base de datos propia. Cuando termina de procesar un lote de datos, notifica al KPI Engine via HTTP para que recalcule los indicadores afectados. Aplica Circuit Breaker en cada llamada a sistemas externos.

6.2.2 Tecnologias y Justificacion

Stack: Java 21 + Spring Boot 3 + Spring Data JPA + Resilience4j

- **Spring Data JPA + Hibernate:** Mapeo objeto-relacional con soporte nativo para JSONB de PostgreSQL, que se usa para almacenar los payloads raw sin forzar un esquema rigido en la columna.
- **Resilience4j:** Implementacion del Circuit Breaker compatible con Spring Boot 3 via anotaciones. Configuracion por sistema fuente (umbral de fallos, tiempo de espera, politica de fallback) en el archivo de propiedades del servicio.
- **Spring WebClient (reactivo):** Para las llamadas HTTP hacia los sistemas fuente y hacia el KPI Engine. Permite configurar timeouts individuales y reintentos sin bloquear hilos.
- **PostgreSQL (events_db):** Almacena el registro de eventos raw, el estado de cada conector y el historial de sincronizaciones. La columna `payload JSONB` permite guardar estructuras heterogeneas sin alterar el esquema.

6.2.3 Diagrama de Flujo: Ciclo de Ingesta con Factory Method y Circuit Breaker

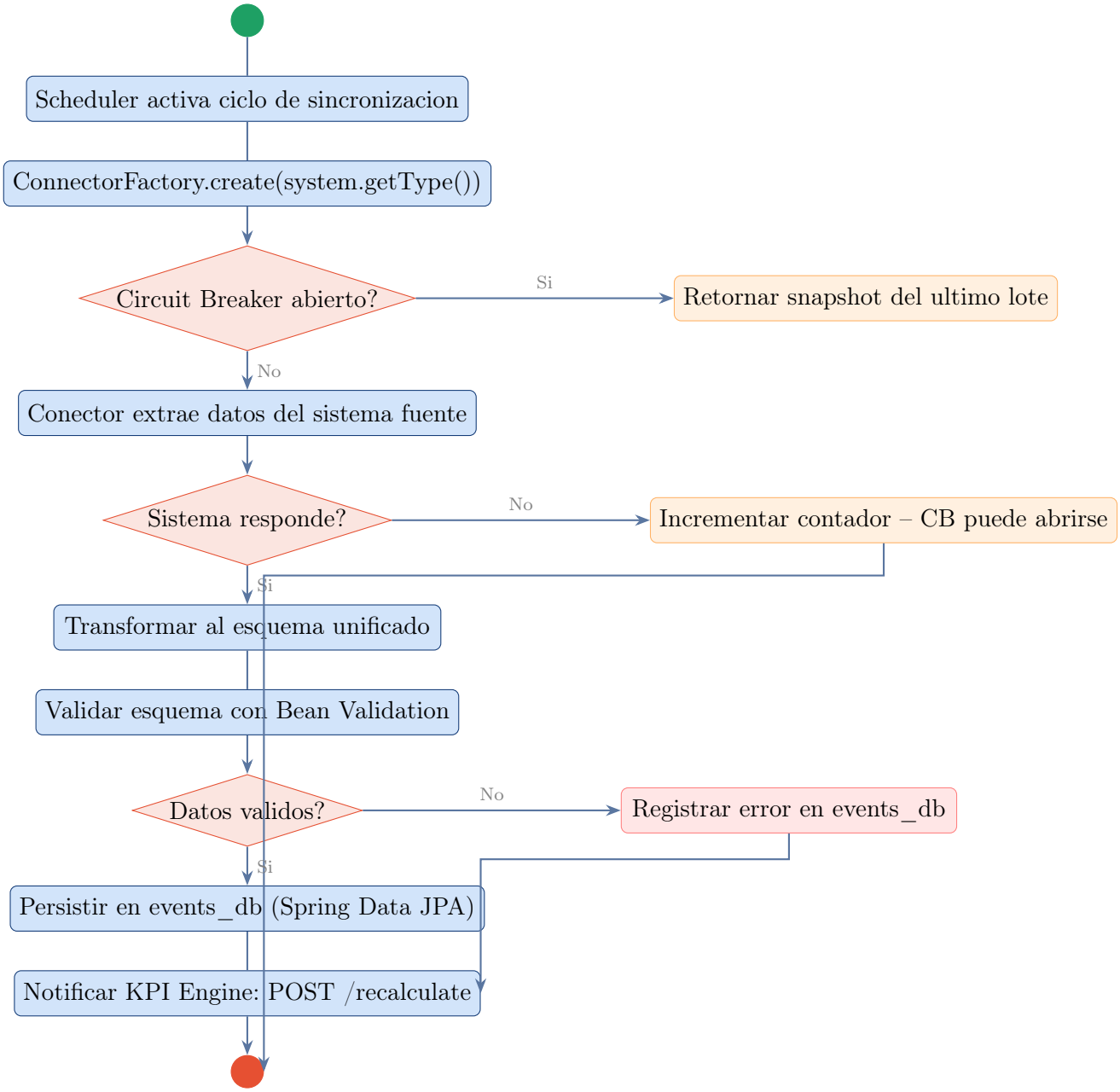

Figura 3: Flujo de ingesta – Factory Method + Circuit Breaker

6.2.4 Dockerfile

Listing 2: Dockerfile - Data Ingestion Service (Java 21)

```
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN ./mvnw package -DskipTests --no-transfer-progress

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/ingestion-service-*.jar app.jar
ENV JAVA_OPTS="-Xms128m -Xmx512m"
EXPOSE 8082
HEALTHCHECK --interval=30s --timeout=5s --start-period=25s \
  CMD wget -qO- http://localhost:8082/actuator/health || exit 1
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

6.3 KPI Engine Service

6.3.1 Responsabilidad

Recibe notificaciones del Data Ingestion Service cuando llegan datos nuevos, consulta los registros afectados, ejecuta las formulas definidas para cada indicador y guarda el resultado calculado en su base de datos. Expone una API REST para que el dashboard y el Report Service consulten los valores de KPI con filtros por sucursal, periodo y area funcional. Tambien detecta desviaciones respecto a umbrales configurados.

6.3.2 Tecnologias y Justificacion

Stack: Java 21 + Spring Boot 3 + Spring Data JPA + Spring Web

- **Java 21 con Virtual Threads (Project Loom):** Los calculos de KPI pueden implicar multiples consultas a la base de datos en paralelo. Los virtual threads de Java 21 permiten escribir codigo bloqueante sin desperdiciar hilos del SO, lo que simplifica la logica de calculo concurrente.
- **Spring Data JPA + consultas JPQL:** Las formulas de KPI se expresan como consultas JPQL con `@Query`, lo que mantiene la logica de calculo proxima a los datos sin introducir un motor de reglas externo.
- **Spring Web (RestController):** Expone los endpoints de consulta de KPI y el endpoint de recalculo que invoca el Data Ingestion Service. Validacion de parametros con Bean Validation.
- **PostgreSQL (kpi_db):** Almacena las definiciones de indicadores (formula, umbral, periodicidad) y el historial de valores calculados. La separacion de tablas `kpi_definition` y `kpi_value` permite agregar nuevos indicadores sin alterar el historico.

6.3.3 Diagrama de Flujo: Calculo de KPI y Deteccion de Alertas

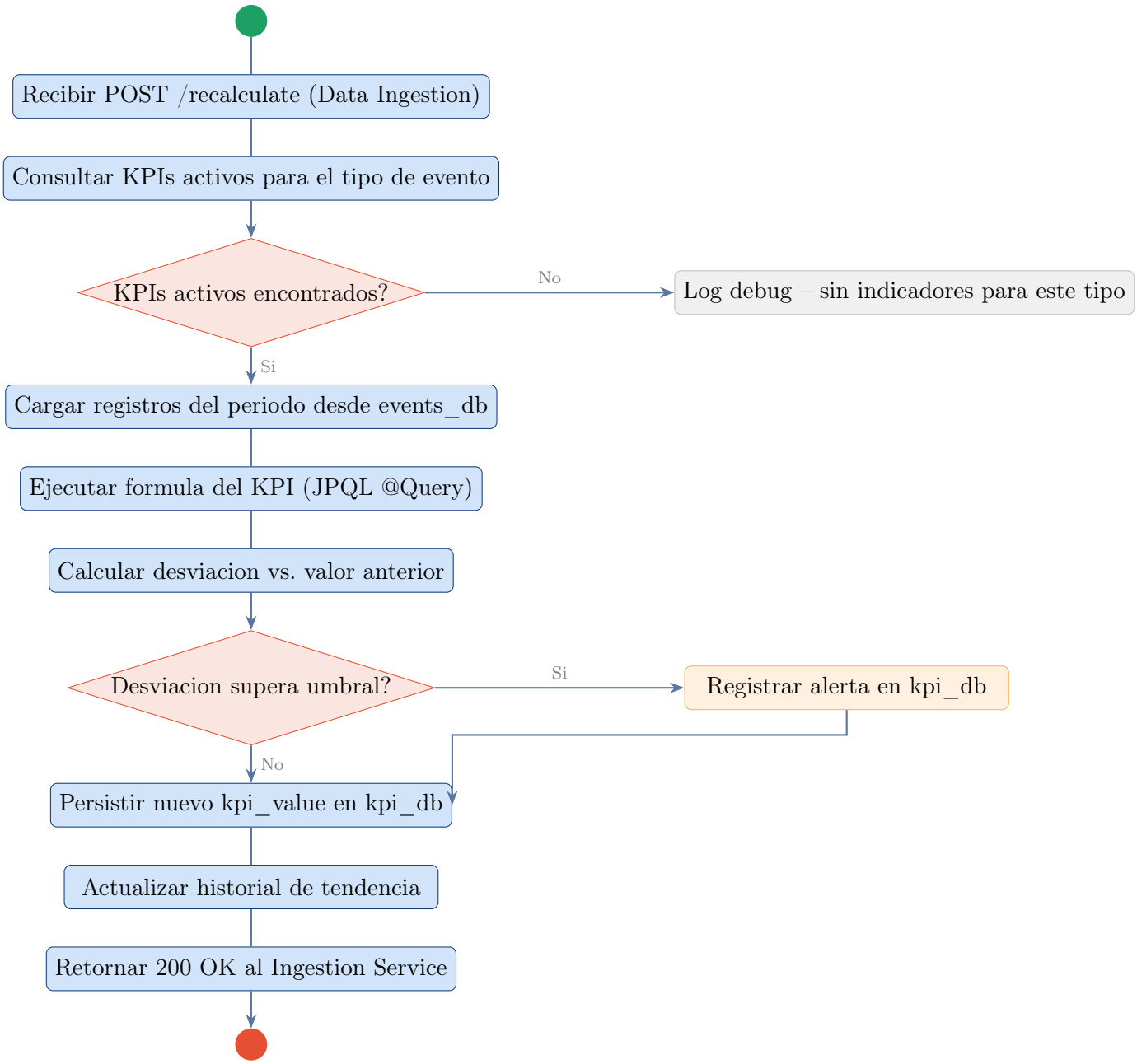

Figura 4: Flujo de calculo de KPI y deteccion de alertas – KPI Engine Service

6.3.4 Dockerfile

Listing 3: Dockerfile - KPI Engine Service (Java 21)

```
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN ./mvnw package -DskipTests --no-transfer-progress

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/kpi-engine-*.jar app.jar
ENV JAVA_OPTS="-Xms128m -Xmx384m"
EXPOSE 8083
HEALTHCHECK --interval=30s --timeout=5s --start-period=25s \
  CMD wget -qO- http://localhost:8083/actuator/health || exit 1
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

6.4 Report Service

6.4.1 Responsabilidad

Genera reportes bajo demanda en formato PDF y Excel. Recibe un request con parametros (periodo, sucursal, metricas), consulta los KPI calculados al KPI Engine Service, arma el reporte con JasperReports y lo almacena temporalmente para exponer una URL de descarga con tiempo de vida configurado.

6.4.2 Tecnologias y Justificacion

Stack: Java 21 + Spring Boot 3 + JasperReports + Apache POI

- **JasperReports:** La libreria de referencia en el ecosistema Java para generacion de reportes empresariales en PDF. Las plantillas `.jrxml` permiten definir layouts complejos (tablas con totales, graficos embebidos, encabezados por sucursal) de forma declarativa. El Report Service compila las plantillas al arrancar y las reutiliza en cada request.
- **Apache POI:** Para exportar reportes en formato Excel (`.xlsx`). Permite crear hojas con estilos de celda, formatos numericos y agrupaciones de filas sin dependencias externas.
- **Spring WebClient:** Para consultar el KPI Engine y el Data Ingestion Service al momento de generar el reporte. Configurado con timeout de 10 segundos; si el KPI Engine no responde, el reporte se genera con los datos disponibles en reports_db.
- **PostgreSQL (reports_db):** Almacena los metadatos de cada reporte generado (parametros, ruta del archivo, usuario solicitante, fecha de expiracion) y el historial de descargas.

6.4.3 Diagrama de Flujo: Generacion de Reporte

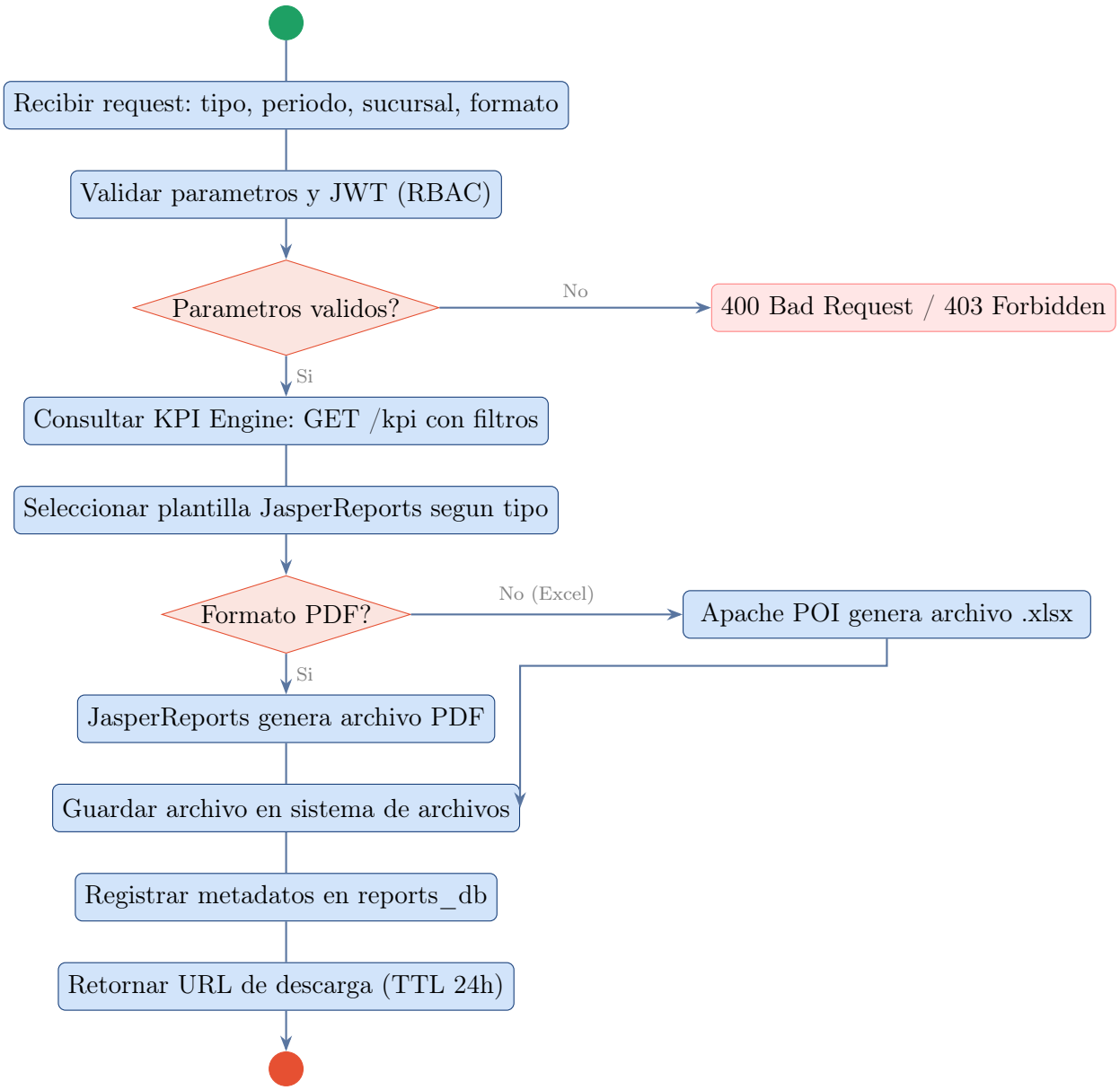

Figura 5: Flujo de generacion de reporte – Report Service

6.4.4 Dockerfile

Listing 4: Dockerfile - Report Service (Java 21 + JasperReports)

```
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN ./mvnw package -DskipTests --no-transfer-progress

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN apk add --no-cache fontconfig ttf-dejavu
COPY --from=builder /app/target/report-service-*.jar app.jar
COPY src/main/resources/reports ./reports
ENV JAVA_OPTS="-Xms128m -Xmx512m"
ENV REPORTS_PATH=/app/reports
EXPOSE 8084
HEALTHCHECK --interval=30s --timeout=5s --start-period=25s \
  CMD wget -qO- http://localhost:8084/actuator/health || exit 1
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

7 Orquestacion con Docker Compose

El archivo `docker-compose.yml` levanta los cuatro servicios junto con sus bases de datos PostgreSQL y la API Gateway. No incluye Redis ni Kafka: esas dependencias se incorporan en la segunda iteración.

Listing 5: docker-compose.yml - Stack completo

```yaml
version: "3.9"

services:

  # - Bases de datos PostgreSQL ------------------------------------------------

  postgres-auth:
    image: postgres:16-alpine
    container_name: auth_db
    environment:
      POSTGRES_DB: auth_db
      POSTGRES_USER: cordillera
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - auth_pg_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U cordillera -d auth_db"]
      interval: 10s
      timeout: 5s
      retries: 5

  postgres-events:
    image: postgres:16-alpine
    container_name: events_db
    environment:
      POSTGRES_DB: events_db
      POSTGRES_USER: cordillera
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - events_pg_data:/var/lib/postgresql/data
    ports:
      - "5433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U cordillera -d events_db"]
      interval: 10s

  postgres-kpi:
    image: postgres:16-alpine
    container_name: kpi_db
    environment:
      POSTGRES_DB: kpi_db
      POSTGRES_USER: cordillera
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - kpi_pg_data:/var/lib/postgresql/data
```

```yaml
    ports:
      - "5434:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U cordillera -d kpi_db"]
      interval: 10s

  postgres-reports:
    image: postgres:16-alpine
    container_name: reports_db
    environment:
      POSTGRES_DB: reports_db
      POSTGRES_USER: cordillera
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - reports_pg_data:/var/lib/postgresql/data
    ports:
      - "5435:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U cordillera -d reports_db"]
      interval: 10s

  # - Microservicios -------------------------------------------------------

  auth-service:
    build:
      context: ./services/auth
      dockerfile: Dockerfile
    container_name: auth_service
    ports:
      - "8081:8081"
    depends_on:
      postgres-auth:
        condition: service_healthy
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-auth:5432/auth_db
      SPRING_DATASOURCE_USERNAME: cordillera
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
      JWT_PRIVATE_KEY: ${JWT_PRIVATE_KEY}
      JWT_EXPIRY_HOURS: 8
    restart: unless-stopped

  data-ingestion:
    build:
      context: ./services/ingestion
      dockerfile: Dockerfile
    container_name: ingestion_service
    ports:
      - "8082:8082"
    depends_on:
      postgres-events:
        condition: service_healthy
      auth-service:
        condition: service_started
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-events:5432/events_db
      SPRING_DATASOURCE_USERNAME: cordillera
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
      KPI_ENGINE_URL: http://kpi-engine:8083
```

```yaml
    CB_FAILURE_THRESHOLD: 5
    CB_WAIT_SECONDS: 60
  restart: unless-stopped

kpi-engine:
  build:
    context: ./services/kpi
    dockerfile: Dockerfile
  container_name: kpi_engine
  ports:
    - "8083:8083"
  depends_on:
    postgres-kpi:
      condition: service_healthy
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-kpi:5432/kpi_db
    SPRING_DATASOURCE_USERNAME: cordillera
    SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
    EVENTS_DB_URL: jdbc:postgresql://postgres-events:5432/events_db
  restart: unless-stopped

report-service:
  build:
    context: ./services/report
    dockerfile: Dockerfile
  container_name: report_service
  ports:
    - "8084:8084"
  depends_on:
    postgres-reports:
      condition: service_healthy
    kpi-engine:
      condition: service_started
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-reports:5432/reports_db
    SPRING_DATASOURCE_USERNAME: cordillera
    SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
    KPI_ENGINE_URL: http://kpi-engine:8083
    REPORTS_TTL_HOURS: 24
  volumes:
    - report_files:/app/generated-reports
  restart: unless-stopped

# - API Gateway ---------------------------------------------------------------

api-gateway:
  image: nginx:alpine
  container_name: api_gateway
  ports:
    - "80:80"
  volumes:
    - ./gateway/nginx.conf:/etc/nginx/nginx.conf:ro
  depends_on:
    - auth-service
    - data-ingestion
    - kpi-engine
    - report-service
  restart: unless-stopped
```

```
volumes:
  auth_pg_data:
  events_pg_data:
  kpi_pg_data:
  reports_pg_data:
  report_files:
```

Por que solo Docker Compose en esta etapa

Kubernetes agrega valor cuando el sistema debe escalar a multiples nodos, manejar cientos de replicas o integrarse con pipelines CI/CD complejos. En esta etapa, el objetivo es que los cuatro servicios funcionen juntos correctamente. Docker Compose cumple ese proposito: levanta el stack completo con un solo comando (`docker compose up`), los logs son accesibles directamente y el ciclo de iteracion es rapido. La configuracion aqui definida es compatible con `kompose` para migrar a Kubernetes en etapas futuras sin reescribir los Dockerfiles.

8 Diagrama BPMN: Flujo Completo entre los Cuatro Servicios

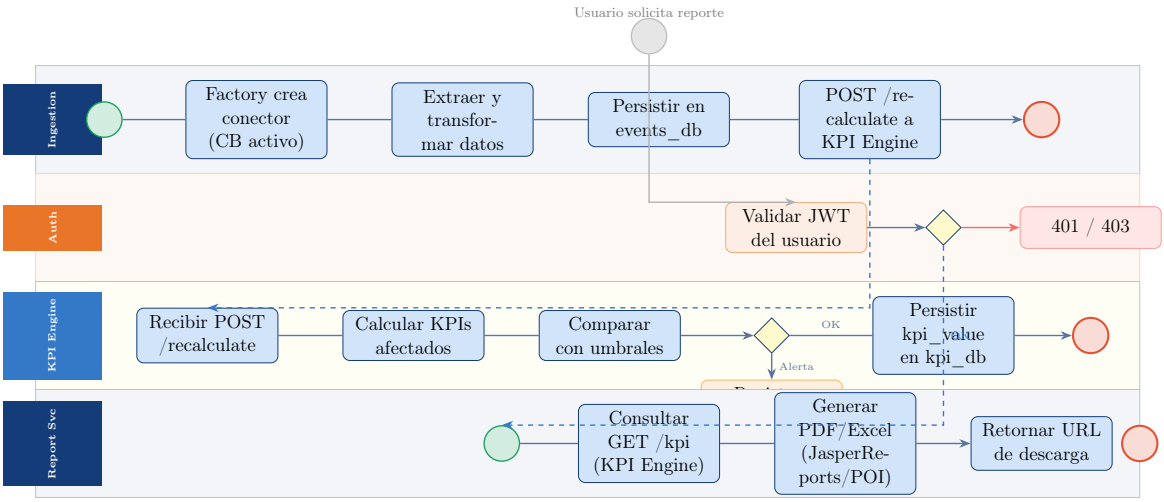

Figura 6: BPMN: flujo completo coordinado entre los cuatro microservicios core

9 Modelo de Datos por Servicio

Cada servicio es duenyo de su base de datos. Ningun servicio accede directamente a la base de datos de otro: si necesita datos de otro servicio, los solicita via API REST.

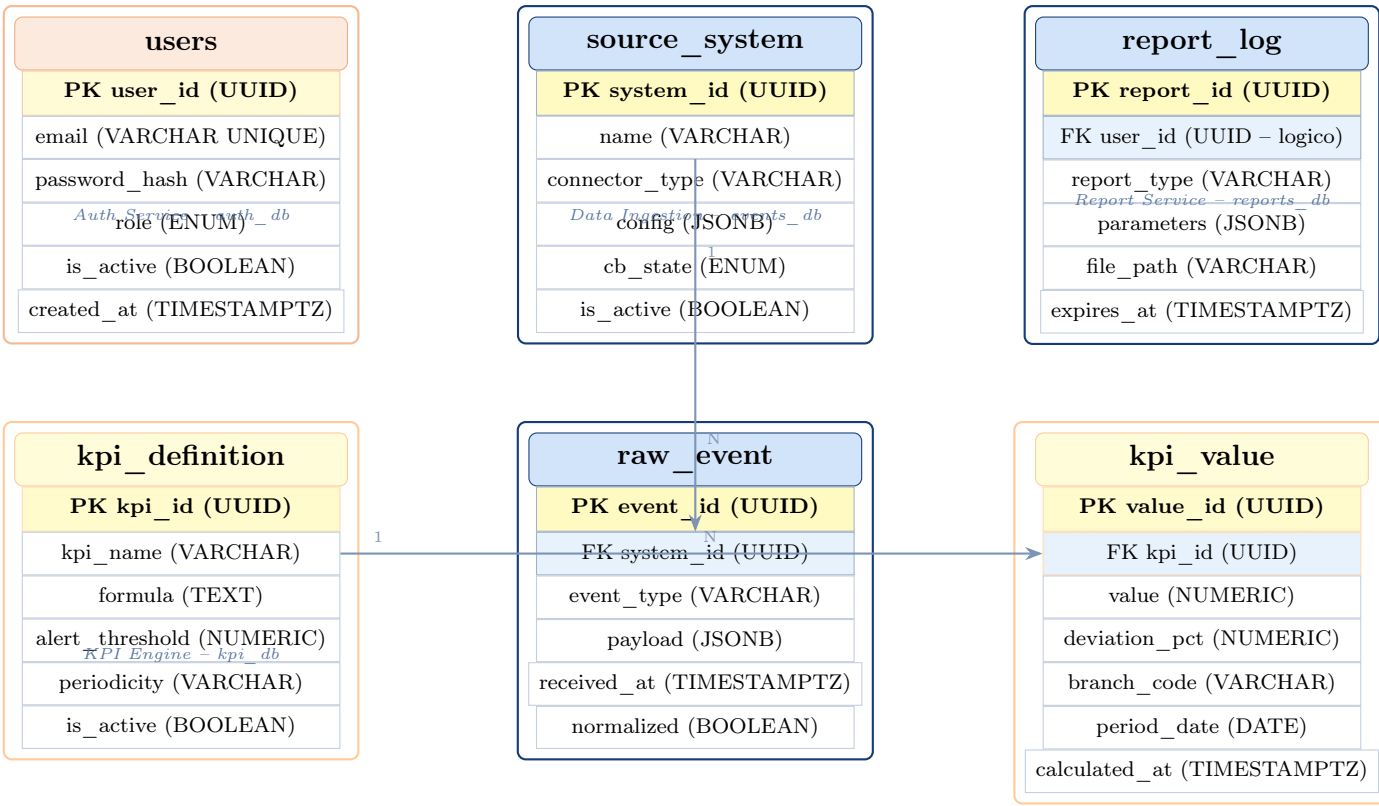

Figura 7: Modelo de datos por servicio – cada uno es duenyo de sus tablas

10 Observabilidad y Monitoreo

La arquitectura incorpora un enfoque inicial de observabilidad para permitir el monitoreo del comportamiento del sistema, la trazabilidad de solicitudes entre servicios y la detección temprana de fallos.

Objetivo de Observabilidad: Proveer visibilidad sobre el estado interno de los microservicios sin necesidad de acceder directamente a sus contenedores, facilitando el debugging, monitoreo y análisis de comportamiento en tiempo real.

10.1 Logs Estructurados

Cada microservicio genera logs en formato estructurado (JSON), lo que permite su procesamiento automatizado por herramientas de análisis y monitoreo.

- Incluyen campos como: `timestamp`, `level`, `service`, `message`.
- Facilitan la integración futura con sistemas como ELK Stack (Elasticsearch, Logstash, Kibana).
- Permiten búsquedas eficientes y correlación entre eventos.

Los logs estructurados eliminan la ambigüedad del texto plano y permiten analizar grandes volúmenes de información sin procesamiento manual.

10.2 Correlation ID

Cada request que entra al sistema recibe un identificador único (`X-Correlation-ID`), que se propaga entre todos los microservicios involucrados.

- Se genera en la API Gateway si no viene en el request.
- Se incluye automáticamente en todos los logs.
- Permite rastrear el flujo completo de una operación entre servicios.

Ejemplo de flujo:

```
Client -> API Gateway (genera ID)
       -> Ingestion Service
       -> KPI Engine
       -> Report Service
```

Beneficio del Correlation ID

Permite identificar rápidamente en qué servicio falló una operación, sin necesidad de revisar logs manualmente en cada contenedor.

10.3 Metricas con Spring Boot Actuator

Todos los microservicios exponen endpoints de monitoreo mediante Spring Boot Actuator.

- `/actuator/health`: Estado del servicio (UP/DOWN)
- `/actuator/metrics`: Métricas de uso (CPU, memoria, requests)
- `/actuator/info`: Información del servicio

Estas métricas permiten:

- Detectar caídas de servicios automáticamente
- Analizar tiempos de respuesta
- Monitorear consumo de recursos por servicio

Configuración base de Actuator

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
```

10.4 Justificacion Tecnica Adicional

La incorporacion de observabilidad desde la primera iteracion evita uno de los problemas mas comunes en arquitecturas de microservicios: la dificultad para diagnosticar fallos distribuidos.

En un sistema monolitico, un error puede rastrearse en un solo log. En cambio, en una arquitectura distribuida como la propuesta, una sola solicitud puede atravesar multiples servicios, bases de datos y puntos de fallo.

Sin mecanismos de observabilidad:

- Los errores se vuelven dificiles de reproducir
- El debugging requiere acceder manualmente a multiples contenedores
- No existe visibilidad del flujo completo de una operacion

La combinacion de logs estructurados, correlation ID y metricas expuestas via Actuator permite:

- Reducir drasticamente el tiempo de diagnostico de errores
- Entender el comportamiento del sistema bajo carga
- Preparar la arquitectura para herramientas avanzadas como tracing distribuido (OpenTelemetry) en futuras iteraciones

Por que incluir observabilidad en esta iteracion

Aunque herramientas avanzadas como Prometheus o Grafana no se implementan en esta etapa, definir desde el inicio los puntos de observabilidad asegura que el sistema pueda escalar sin necesidad de refactorizar la forma en que se generan logs o metricas.

11 Justificacion de Decisiones Tecnicas

11.1 Por que Java 21 con Spring Boot 3 para todos los servicios

Usar el mismo lenguaje y framework en los cuatro servicios no es la unica opcion posible, pero si la mas practica para un equipo de dos personas. El costo de cambiar de contexto entre lenguajes se subestima: cuando hay un bug a las 11 de la noche antes de la entrega, tener todo en Java significa que cualquiera de los dos puede revisar cualquier servicio.

Spring Boot 3 tiene soporte nativo para los tres patrones requeridos: Spring Data JPA implementa el Repository Pattern sin codigo adicional, el factory se construye con componentes Spring estandar, y Resilience4j se integra via anotaciones sin configuracion XML. El framework elimina el boilerplate sin ocultar lo que esta pasando, lo que facilita la documentacion y el aprendizaje en contexto academico.

Java 21 agrega los Virtual Threads (Project Loom), que son especialmente utiles en el KPI Engine cuando necesita hacer multiples consultas a la base de datos en paralelo para calcular indicadores de distintas areas. El codigo queda igual que el codigo bloqueante tradicional pero sin desperdiciar hilos del sistema operativo.

11.2 Por que PostgreSQL para todos los servicios

Cada servicio tiene su propia base de datos PostgreSQL. No es una base de datos compartida: son cuatro instancias separadas en contenedores separados. La razon de usar el mismo motor es operacional: un solo tipo de base de datos significa un solo tipo de backup, un solo tipo de monitoreo y una sola curva de aprendizaje para el equipo.

PostgreSQL especificamente porque soporta el tipo `JSONB` nativamente, que es fundamental para el Data Ingestion Service: los payloads que llegan de cinco sistemas distintos no tienen el mismo esquema, y guardarlos en JSONB permite almacenarlos sin forzar una estructura rigida mientras se normaliza el resto de los campos.

11.3 Por que comunicacion sincrona HTTP y no Kafka en esta etapa

Kafka es la tecnologia correcta cuando la comunicacion entre servicios necesita ser asincrona, duradera y desacoplada en tiempo. En esta etapa, el flujo es simple: el Data Ingestion Service termina de procesar un lote y le avisa al KPI Engine que recalcule. Eso funciona perfectamente con una llamada HTTP sincrona.

Introducir Kafka ahora agregaria cuatro cosas a gestionar (Zookeeper, Kafka broker, topics, consumer groups) antes de que el flujo basico funcione. en la segunda iteración es el momento correcto para esa transicion, cuando los servicios ya esten estables y los tests de integracion pasen. La arquitectura HTTP actual no necesita reescritura para incorporar Kafka: solo se agrega un producer en el Data Ingestion Service y un consumer en el KPI Engine, y la llamada HTTP pasa a ser un evento.

11.4 Por que JasperReports y no Puppeteer

JasperReports es Java puro, lo que significa que no agrega dependencias de runtime externas al contenedor. No necesita Chrome ni Node.js instalados. Las plantillas `.jrxml` se compilan al arrancar el servicio y se reutilizan en cada request, lo que hace la generacion de reportes rapida incluso bajo carga. Para un equipo que trabaja en Java, mantener las plantillas en XML es mas familiar que mantener templates HTML/CSS que se convierten a PDF via browser headless.

12 Variables de Entorno y Configuracion

El archivo `.env` en la raiz del proyecto define las variables sensibles que Docker Compose inyecta en cada contenedor. Nunca se versiona en el repositorio.

Listing 6: .env - Variables de entorno (no versionar)

```
# Contrasena compartida para las bases de datos PostgreSQL
DB_PASSWORD=cambia_esto_en_produccion

# Clave privada RSA para firmar tokens JWT (formato PEM, sin saltos de linea)
JWT_PRIVATE_KEY=-----BEGIN RSA PRIVATE KEY-----...

# Perfil de Spring Boot activo
SPRING_PROFILES_ACTIVE=dev
```

Cada servicio declara sus propias variables de entorno en `application.yml` con valores por defecto para desarrollo local:

Listing 7: application.yml - KPI Engine Service (ejemplo)

```
server:
  port: 8083

spring:
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5434/kpi_db}
    username: ${SPRING_DATASOURCE_USERNAME:cordillera}
    password: ${SPRING_DATASOURCE_PASSWORD:dev_password}
  jpa:
    hibernate:
      ddl-auto: validate
    show-sql: false

kpi:
  engine:
    events-db-url: ${EVENTS_DB_URL:jdbc:postgresql://localhost:5433/events_db}

resilience4j:
  circuitbreaker:
    instances:
      kpiCalculation:
        failure-rate-threshold: 50
        wait-duration-in-open-state: 30s
        sliding-window-size: 10

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
```

13 Conclusion

Los cuatro servicios cubren el ciclo completo que Grupo Cordillera necesita en esta primera etapa: entrada de datos desde los sistemas existentes, calculo de indicadores, control de acceso y generacion de reportes para la alta direccion.

El stack es deliberadamente uniforme: Java 21 con Spring Boot 3 en todos los servicios, PostgreSQL en todas las bases de datos, Docker Compose como unica capa de orquestacion. Eso reduce la carga de aprendizaje, facilita el debugging entre servicios y deja la arquitectura lista para incorporar los componentes de la Iteracion 2 (Kafka para comunicacion asincrona) sin reescribir lo que ya funciona.

Los tres patrones requeridos estan implementados: Repository Pattern via Spring Data JPA en cada servicio, Factory Method en el Data Ingestion Service para los conectores, y Circuit Breaker via Resilience4j protegiendo las llamadas a sistemas externos.